
Apache Wave (incubating) Protocol Documentation

Release 0.4

The Apache Wave Foundation

August 22, 2015

1	FAQ	3
2	Google Wave Operational Transformation	5
2.1	Executive Summary	5
2.2	Introduction	5
2.3	Wave Extensions to Operational Transformation	6
2.4	Wavelet Operations	8
2.5	References	10
3	Google Wave Federation Architecture	11
3.1	Wave Providers	11
3.2	Waves, Wavelets, and Identifiers	12
3.3	Wave Service Architecture	12
3.4	Federation Gateway and Federation Proxy	13
3.5	Protocol	16
3.6	References	17
4	Google Wave Client-Server Protocol Whitepaper	19
4.1	Editorial Notes	19
4.2	Introduction	19
4.3	Background	19
4.4	Scope	19
4.5	Data Model	20
4.6	Operation	20
4.7	Specific Flows	22
4.8	Serializing Protocol Buffers as JSON	22
4.9	Security	23
4.10	Client-Server Protocol Buffers	24
4.11	Example Client-Server Flow	24
4.12	Appendix A - Open Source Implementation Notes	24
5	Access Control in Google Wave	25
5.1	Executive summary	25
5.2	Groups	27
6	Google Wave Attachments	31
6.1	High level summary	31
6.2	Architecture	31
6.3	Schema	32
6.4	Attachment Creation	33
6.5	Uploading	35

6.6	Attachment / Thumbnail download	36
6.7	Authentication / Authorization	36
6.8	Duplicate elimination	36
6.9	References	36

The following Papers talk about the protocols and foundations of Apache Wave. Where applicable the original authors have been credited for the publication and each document has had revision by the Apache Software Foundation.

Apache Wave has been through many revisions, first Google Wave then WIAB (Wave in a Box) until it joined the Apache Incubator. The structure of wave is complicated and has many moving parts and specification, these papers will include high level technical information about how Apache Wave comes together.

7.. Licensed to the Apache Software Foundation (ASF) under one or more contributor license agreements. See the NOTICE file distributed with this work for additional information regarding copyright ownership. The ASF licenses this file to you under the Apache License, Version 2.0 (the “License”); you may not use this file except in compliance with the License. You may obtain a copy of the License at

CHAPTER
ONE

FAQ

GOOGLE WAVE OPERATIONAL TRANSFORMATION

Authors David Wang, Alex Mah, Soren Lassen

Version 1.1 - July 2010

This whitepaper is part of a series. All of the whitepapers can be found on [Google Wave Federation Protocol site](#).

Waves are hosted, structured documents that allow seamless and low latency concurrent modifications. To provide this live experience, Google Wave uses the Operational Transformation (OT) framework of concurrency control.

2.1 Executive Summary

Collaborative document editing means multiple editors are able to edit a shared document at the same time. It is live and concurrent when a user can see the changes another person is making, keystroke by keystroke. Google Wave offers live concurrent editing of rich text documents.

The result is that Google Wave allows for a very engaging conversation where you can see what the other person is typing, character by character, much like how you would converse in a cafe. This is very much like instant messaging except you can see what the other person is typing, live. Google Wave also allows for a more productive collaborative document editing experience, where people don't have to worry about stepping on each others toes and still use common word processor functionalities such as bold, italics, bullet points, and headings.

Waves are more than just rich text documents. In fact, Google Wave's core technology allows live concurrent modifications of structured documents which can be used to represent any structured content including system data that is shared between clients and backend systems.

To achieve these goals, Google Wave uses a concurrency control system based on Operational Transformation.

2.2 Introduction

Operational transformation (OT) is a theoretical framework of concurrency control that has been continuously researched in the context of group editing for more than 10 years. This document does not describe the basic theory of OT and assumes the reader understands OT. The reader is encouraged to read the documents in the reference section for background.

In short, Wave OT replicates the shared document at all sites and allows any user to edit any part of the document at any time. Local editing operations are executed without being delayed or blocked. Remote operations are transformed before execution. The lock-free, non-blocking property of OT makes the local response time insensitive to networking latencies. These properties of OT play a big part in providing the Optimistic User Interface (UI) of Wave. Optimistic UI means user actions are executed and displayed locally to the user immediately without waiting for the server to respond.

The starting point for Wave OT was the paper “High-latency, low-bandwidth windowing in the Jupiter collaboration system”. Like the Jupiter system described by the paper, Google Wave also implements a client and server based OT system. The reader is again encouraged to read this paper for background.

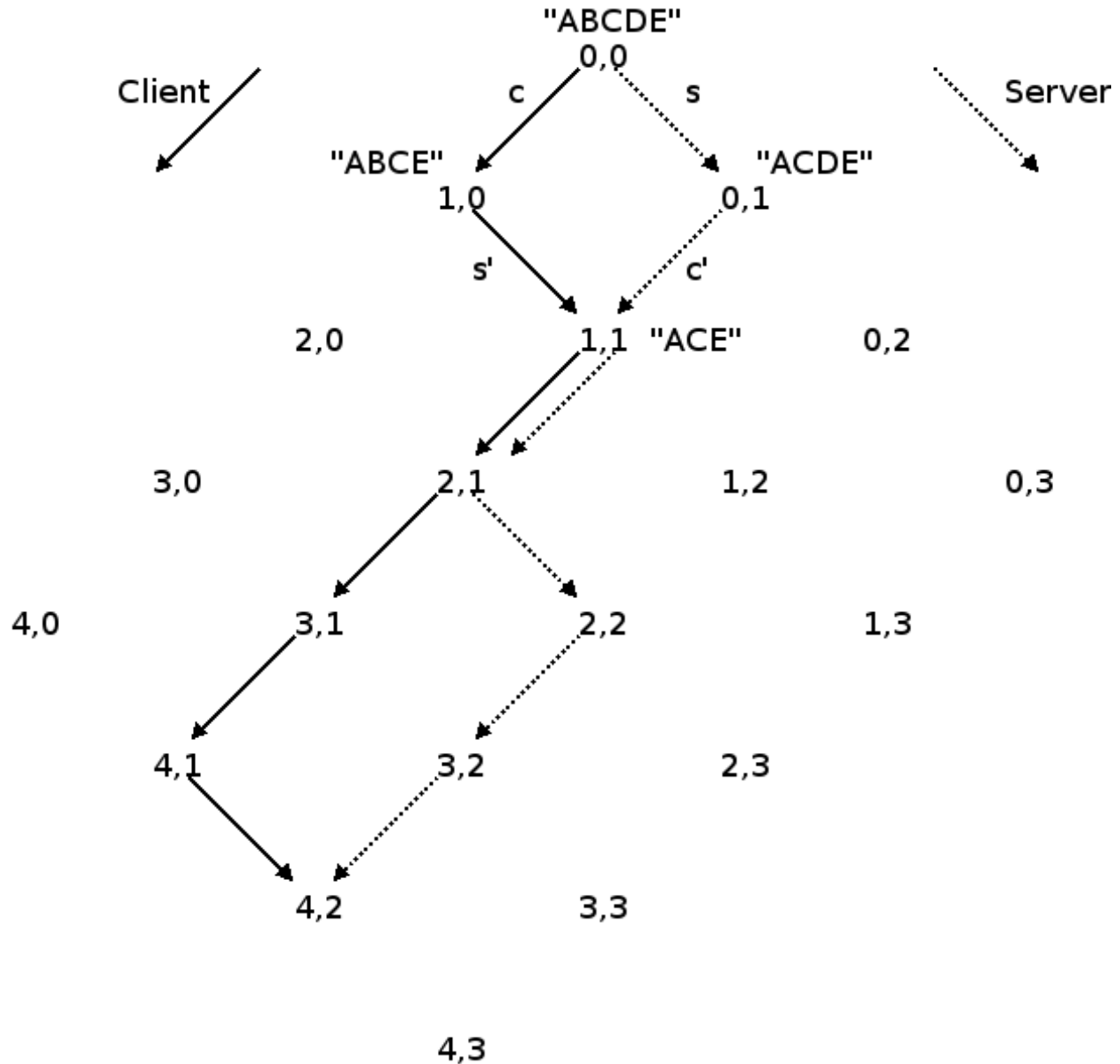
This document describes the extensions Google Wave made to the basic theory of OT.

2.3 Wave Extensions to Operational Transformation

A wave is a collection of wavelets. A wavelet contains a collection of documents. A document consists of a structured, XML-like document and some annotations. A wavelet is where concurrent modification takes place. A wavelet is the object on which OT is applied.

2.3.1 Clients wait for acknowledgement from server before sending more operations

To recap, under the basic theory of OT, a client can send operations sequentially to the server as quickly as it can. The server can do the same. This means the client and server can traverse through the state space via different OT paths to the same convergent state, depending on when they receive the other parties’ operations. See diagram below.



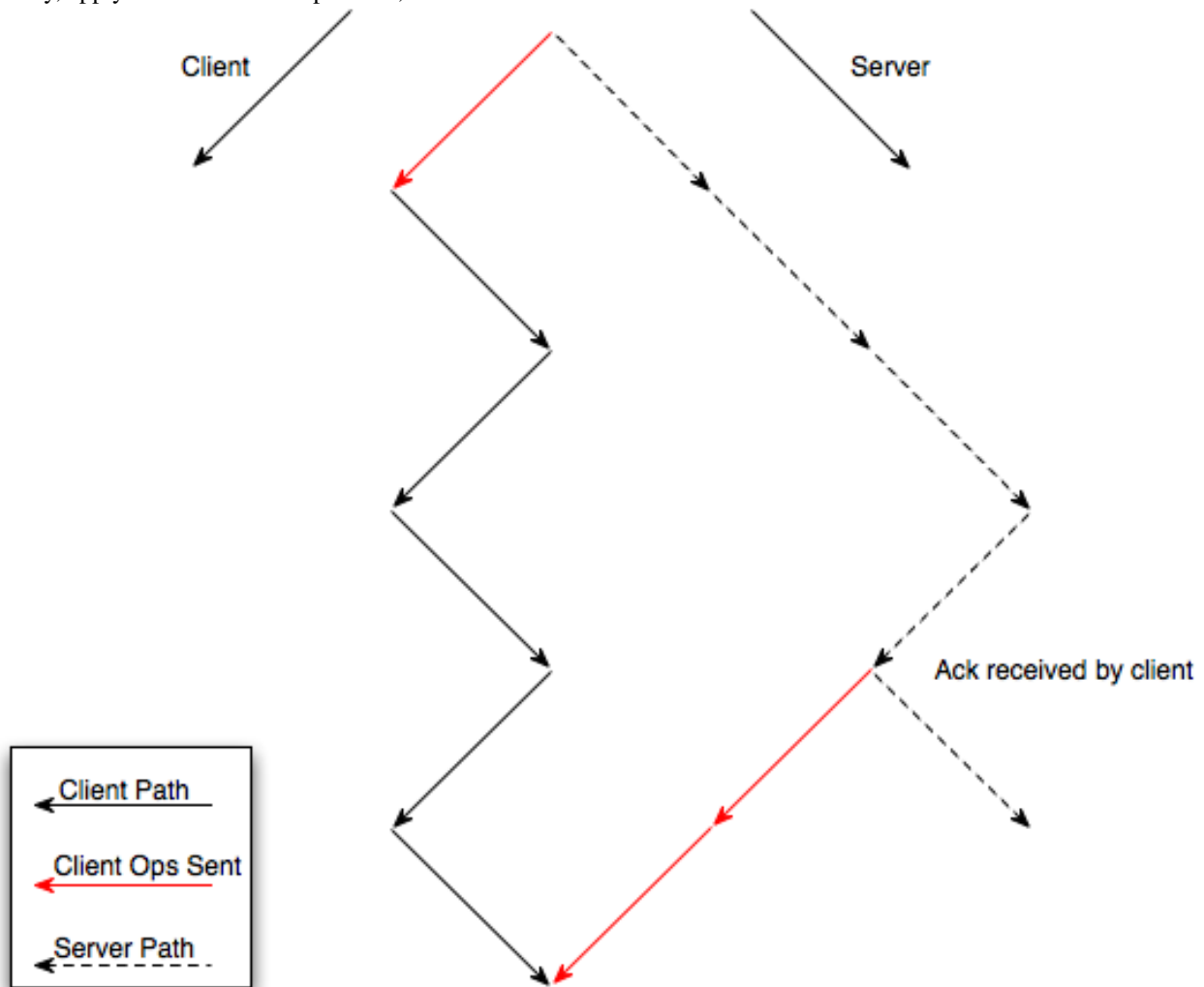
When you have multiple clients connected to the server, every client and server pair have their own state space. One shortcoming of this is the server needs to carry a state space for every connected client, which can be memory-intensive. In addition, this complicates the server algorithm by requiring it to convert clients' operations between state spaces.

Having a simple and efficient server is important in making waves reliable and scalable. With this goal, Wave OT modifies the basic theory of OT by requiring the client to wait for acknowledgement from the server before sending more operations. When a server acknowledges a client's operation, it means the server has transformed the client's operation, applied it to the server's copy of the wavelet and broadcast the transformed operation to all other connected clients. Whilst the client is waiting for the acknowledgement, it caches operations produced locally and sends them in bulk later.

With the addition of acknowledgements, a client can infer the server's OT path. We call this the inferred server path. By having this, the client can send operations to the server that are always on the server's OT path.

This has the important benefit that the server only needs to have a single state space, which is the history of operations it has applied. When it receives a client's operation, it only needs to transform the operation against the operation

history, apply the transformed operation, and then broadcast it.



One trade-off of this simplification is that a client will see chunks of operations from another client in intervals of approximately one round trip time to the other client. We believe the server-side benefits make this a worthwhile trade-off.

2.4 Wavelet Operations

Wavelet operations consist of document operations, for modifying documents, and non-document operations, for tasks such as adding or removing a wavelet participant. We'll focus on document operations here.

2.4.1 Document Support

A document operation has a streaming interface, similar to an `XMLStreamWriter` or a SAX handler. The document operation consists of a sequence of ordered document mutations. The mutations are applied in sequence as you traverse the document linearly.

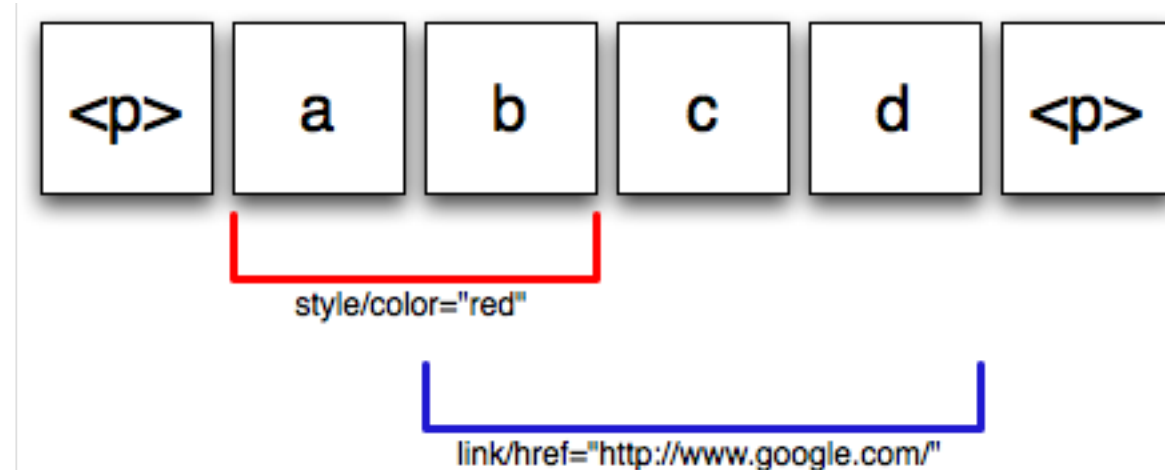
Designing document operations in this manner makes it easier to write the transformation function and composition function described later.

A wave document can be regarded as a single document operation that can be applied to the empty document.

In Google Wave, every character, start tag or end tag in a document is called an item. Gaps between items are called positions. Position 0 is before the first item. A document operation can contain mutations that reference positions. For example, a “Retain” mutation specifies how many positions to skip ahead in the document before applying the next mutation.

`<blip>` _{0 1} `<p>` _{2 3 4 5 6 7 8 9} `example` ₁₀ `</p>` ₁₁ `</blip>`

Wave document operations also support annotations. An annotation is some meta-data associated with an item range, i.e., a start position and an end position. This is particularly useful for describing text formatting and spelling suggestions, as it does not unnecessarily complicate the underlying structured document format.



Wave document operations consist of the following mutation components:

- retain
- insert characters
- insert element start
- insert element end
- delete characters
- delete element start
- delete element end
- replace attributes
- update attributes
- annotation boundary

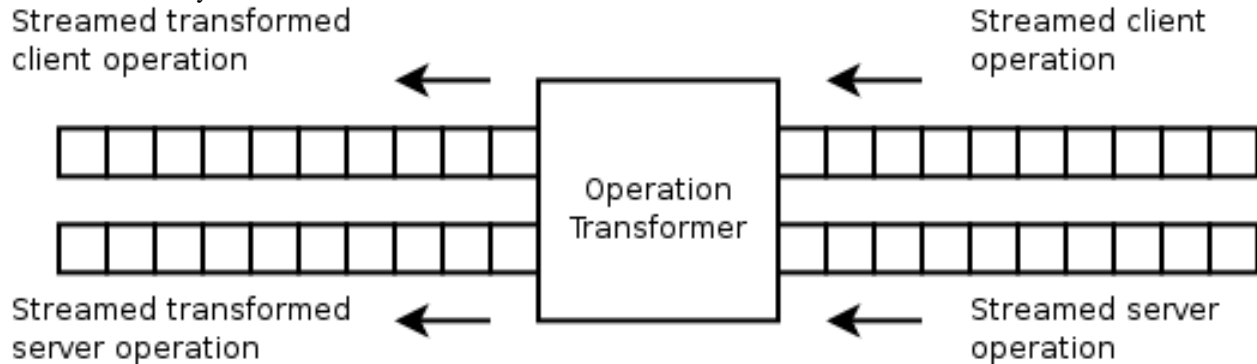
The following is a more complex example document operation.:

```
retain 3
insert element start with tag "p" and no attributes
insert characters "Hi there!"
insert element end
retain 5
delete characters 4
retain 2
```

From this, one can see how an entire document can be represented as a single document operation.

2.4.2 Transformation Function

Representing document operations using a stream interface has the benefit that it makes processing operations in a linear fashion easy.

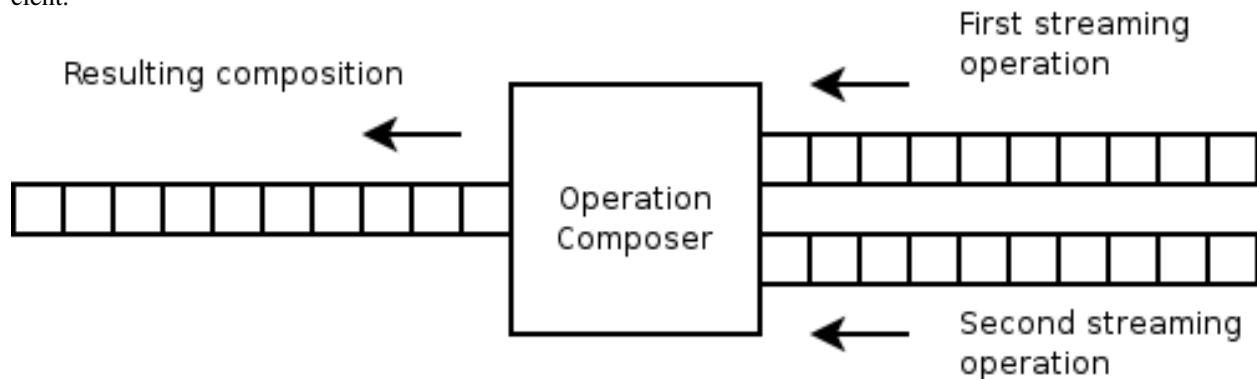


The operation transformer works by taking two streaming operations as input, simultaneously processing the two operations in a linear fashion, and outputting two streaming operations. This stream-style processing ensures that transforming a pair of very large operations is efficient.

2.4.3 Composition

The document operations have been engineered so that they can be composed together and the composition of any two document operations that can be composed together is itself a single document operation.

Furthermore, the composition algorithm processes operations as linear streams, so the composition algorithm is efficient.



The composition BA has the property that $(BA)(d) = B(A(d))$ for all documents d on which A can be applied.

While a Wave client awaits server acknowledgement, it composes all its pending operations. This reduces the number of operations to transform and send.

2.5 References

“Operational transformation”. In Wikipedia, the free encyclopedia, May 28, 2009. http://en.wikipedia.org/wiki/Operational_transformation

David A. Nichols, Pavel Curtis, Michael Dixon, and John Lamping: [High-latency, low-bandwidth windowing in the Jupiter collaboration system](#), UIST '95: Proceedings of the 8th annual ACM symposium on User interface and software technology, pp.111-120. ACM, 1995.

GOOGLE WAVE FEDERATION ARCHITECTURE

Authors Soren Lassen, Sam Thorogood

Version 1.0 - May 2009

This whitepaper is part of a series. All of the whitepapers can be found on [Google Wave Federation Protocol site](#).

Google Wave is a new communication and collaboration platform based on hosted documents (called waves) supporting concurrent modifications and low-latency updates. This platform enables people to communicate and work together in new, convenient and effective ways. We will offer these benefits to users of <http://wave.google.com> and we also want to share them with everyone else by making waves an open platform that everybody can share. We welcome others to run wave servers and become wave providers, for themselves or as services for their users, and to “federate” waves, that is, to share waves with each other and with <http://wave.google.com>. In this way users from different wave providers can communicate and collaborate using shared waves. We are introducing the Google Wave Federation Protocol for federating waves between wave providers on the Internet.

This document gives an overview of how various elements of Google Wave technology – data model, operational transformation, and client-server protocol – are used together to run a wave service, and how wave service providers communicate using the Google Wave Federation Protocol with its cryptographic measures to prevent spoofing. All these elements are described in more depth in accompanying documents on this site and the reader is encouraged to consult them for more details. The focus of this document is federation, which involves the server-server wave federation protocol, and does not cover the client-server protocol between the clients and the wave server of a wave provider. Nonetheless, this document is far from an exhaustive account of wave federation. In particular, [attachments](#), groups, contacts, and presence are important elements of wave federation that are not covered herein.

3.1 Wave Providers

The wave federation protocol enables everyone to become a wave provider and share waves with others. For instance, an organization can operate as a wave provider for its members, an individual can run a wave server as a wave provider for a single user or family members, and an Internet service provider can run a wave service as another Internet service for its users as a supplement to email, IM, ftp, etc. In this model, wave.google.com is one of many wave providers.

A wave provider is identified by its Internet domain name(s).

Wave users have wave addresses which consist of a user name and a wave provider domain in the same form as an email address, namely `<username>@<domain>`. Wave addresses can also refer to groups, robots, gateways, and other services. A group address refers to a collection of wave addresses, much like an email mailing list. A robot is an automated participant on a wave (see the [Robots API](#)). Examples are translation robots and chess game robots. A gateway translates between waves and other communication and sharing protocols such as email and IM. In the remainder we ignore addresses that are services, including robots and gateways; they are treated largely the same as users with respect to federation.

Wave users access all waves through their wave provider. If a wave has participants from different wave providers, their wave providers all maintain a copy of the wave and serve it to their users on the wave. The wave providers share

updates to the wave with each other using the wave federation protocol which we describe below. For any given wave user, it is the responsibility of the wave provider for the user's domain to authenticate the user (using cookies and passwords, etc) and perform local access control.

3.2 Waves, Wavelets, and Identifiers

A wave consists of a set of wavelets. When a user has access to a wavelet, that user is called a participant of that wavelet. Each wavelet has a list of participants, and a set of documents that make up its contents. Different wavelets of a wave can have different lists of participants. Copies of a wavelet are shared across all of the wave providers that have at least one participant in that wavelet. Amongst these wave providers, there is a designated wave provider that has the definitive copy of that wavelet. We say that this particular provider is hosting that wavelet.

When a user opens a wave, a view of the wave is retrieved, namely the set of wavelets in the wave that the user is a participant of (directly, or indirectly via group membership). In general, different users have different wave views for a given wave. For example, per-user data for a user in a wave, such as the user's read/unread state for the wave, is stored in a user-data wavelet in the wave with the user as the only participant. The user-data wavelet only appears in this user's wave view. Another example is a private reply within a wave, which is represented as a wavelet with a restricted participant list. The private reply wavelet is only in the wave views of the restricted list of users.

A wave is identified by a globally unique wave id, which is a pair of a domain name and an id string. The domain names the wave provider where the wave originated.

A wavelet has a wavelet id which is unique within its wave. Like a wave id, a wavelet id is a pair of a domain name and an id string. The domain name in the wavelet id plays a special role: It names the wave provider that hosts the wavelet. A wavelet is hosted by the wave provider of the participant who creates the wavelet. The wave provider who hosts a wavelet is responsible both for operational transformation and application of wavelet operations to the wavelet and for sharing the updates with the wave providers of all the wavelet participants, as described in the Wave Servers section below. The updates are wavelet operations and concurrent updates are resolved using operational transformation.

Wavelets in the same wave can be hosted by different wave providers. For example, a user-data wavelet is always hosted by the user's wave provider, regardless of where the rest of the wave is hosted. Indeed, user-data is not federated, i.e., not shared with other wave providers. Another example is a private reply wavelet. A particularly simple instance of this is when all the participants of the private reply are from the same wave provider. Then this wave provider will not share the private reply wavelet with other wave providers, regardless of where the other wavelets in the wave are hosted.

3.3 Wave Service Architecture

A wave provider operates a wave service on one or more networked servers. The central pieces of the wave service is the wave store, which stores wavelet operations, and the wave server, which resolves wavelet operations by operational transformation and writes and reads wavelet operations to and from the wave store. Typically, the wave service serves waves to users of the wave provider which connect to the wave service frontend (see the [Google Wave Data Model and Client-Server Protocol](#)), and we shall assume this in the following description of the wave service architecture. More importantly, for the purpose of federation, the wave service shares waves with participants from other providers by communicating with these wave provider's servers. The wave service uses two components for this, a federation gateway and a federation proxy. They are described in the next section.

A wave provider's wave server serves wave views to local participants, i.e., participants from its domain. As described earlier, copies of a wavelet are distributed to all wave providers that have participants in that wavelet. Copies of a wavelet at a particular provider can either be local or remote. We use the term "local wavelet" and "remote wavelet" to refer to these two types of wavelet copies (in both cases, we are referring to the wavelet copy, and not the wavelet). A wave view can contain both types of wavelet copies simultaneously.

At a particular wave provider, local wavelets are those created at that provider, namely by users who belong to the wavelet provider. The wave server is responsible for processing the wavelet operations submitted to the wavelet by local participants and by remote participants from other wave providers. The wave server performs concurrency control by ordering the submitted wavelet operations relative to each other using operational transformation. It also validates the operations before applying them to a local wavelet.

Remote wavelets are hosted by other wave providers. The wave server maintains cached copies locally and updates them with wavelet operations that it gets from the hosting wave providers. When a local participant submits a wavelet operation to a remote wavelet, the wave server forwards the operation to the wave server of the hosting provider. When the transformed and applied operation is echoed back, it is applied to the cached copy. Read access to local participants is done from the cached copy without a round trip to the hosting wave provider.

Local and remote wavelets are all stored in the wave server's persistent wave store.

We say that a wave provider is “upstream” relative to its local wavelets and that it is “downstream” relative to its remote wavelets.

3.4 Federation Gateway and Federation Proxy

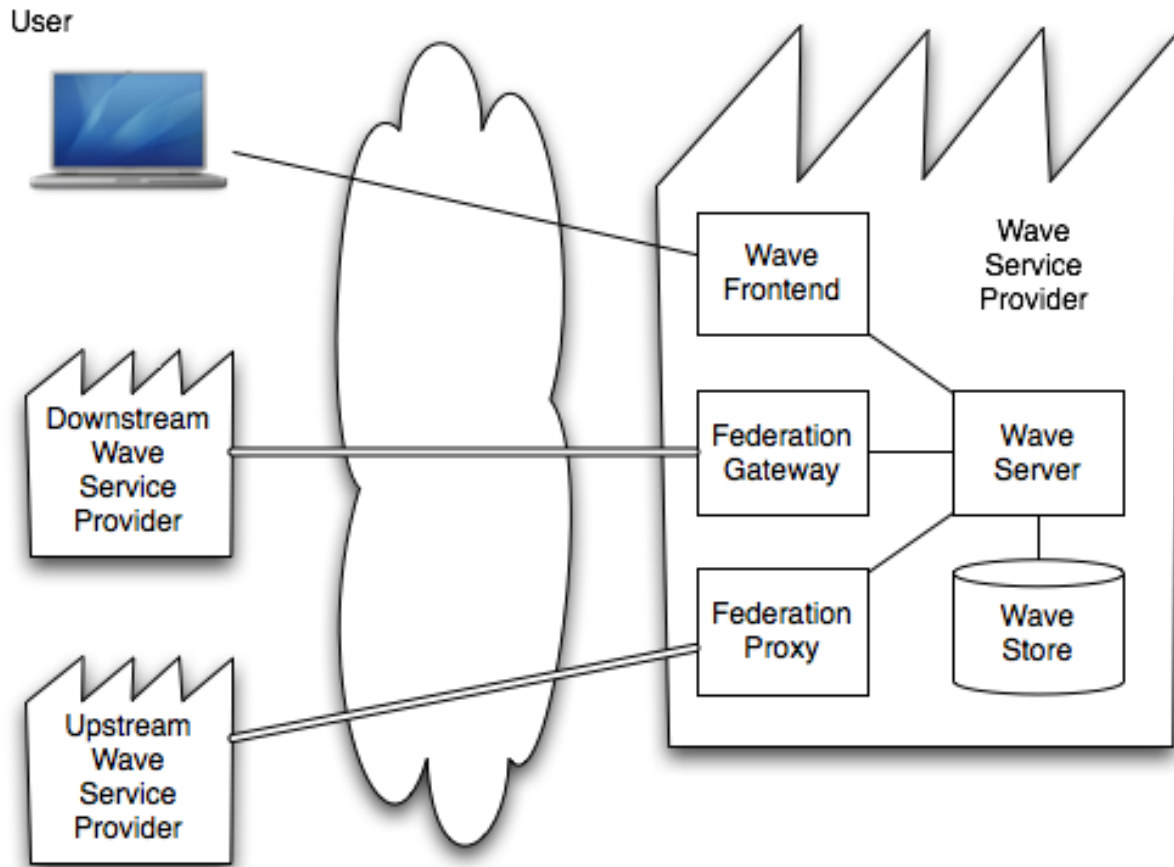
The wave service uses a federation gateway and a federation proxy component to communicate with other wave providers.

The federation gateway communicates local wavelet operations, i.e., operations on local wavelets: * It pushes new wavelet operations that are applied to a local wavelet to the wave providers of any remote participants. * It satisfies requests for old wavelet operations. * It processes wavelet operations submission requests.

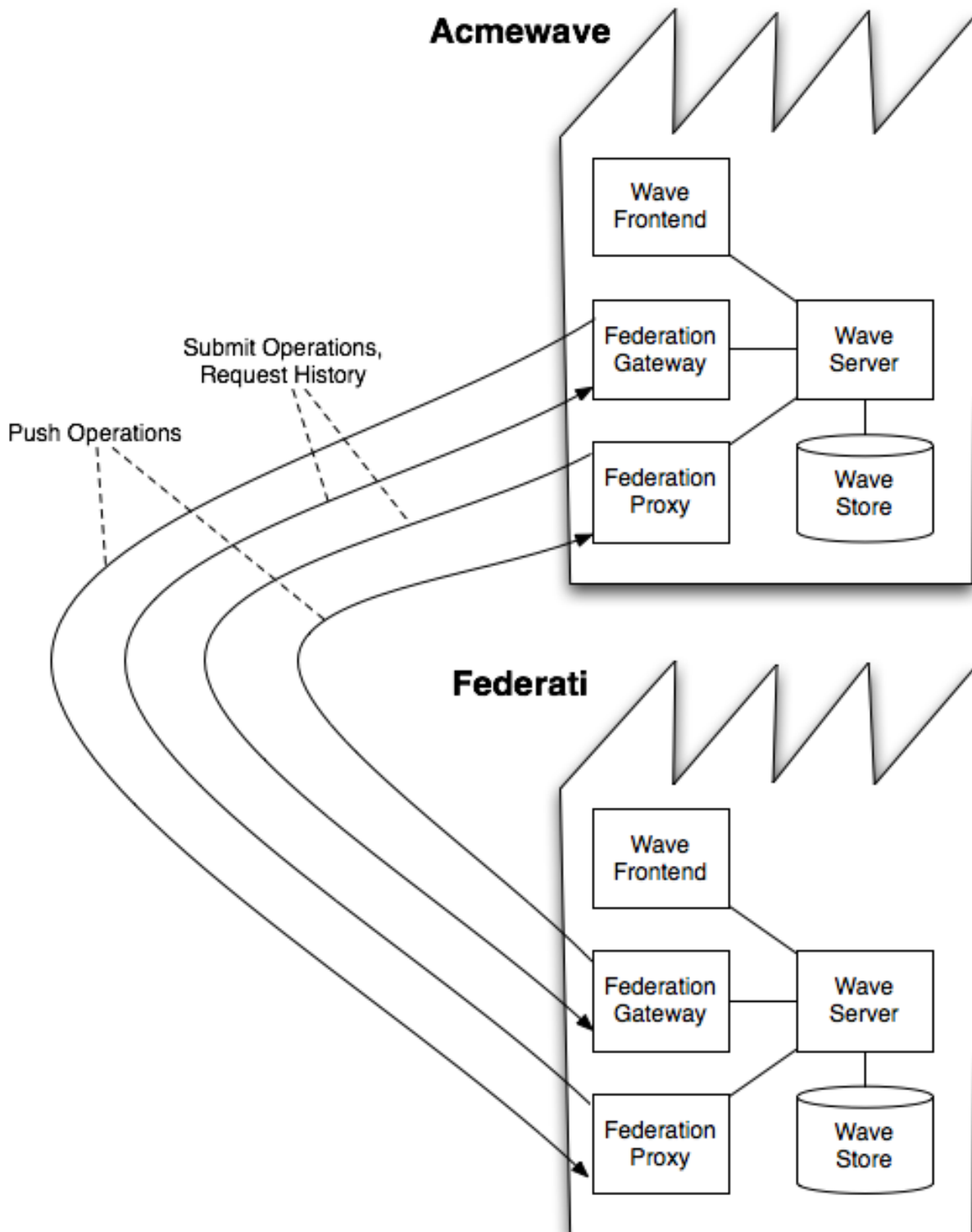
The federation proxy communicates remote wavelet operations and is the component of a wave provider that communicates with the federation gateway of remote providers: * It receives new wavelet operations pushed to it from the wave providers that host the remote wavelets. * It requests old wavelet operations from the hosting wave providers. * It submits wavelet operations to the hosting wave providers.

An upstream wave provider's federation gateway connects to a downstream wave provider's federation proxy to push wavelet operations that are hosted by the upstream wave provider.

The federation protocol has the following mechanisms to make operation delivery from gateway to proxy reliable. The federation gateway maintains (in persistent storage) a queue of outgoing operations for each remote domain. Operations are queued until their receipt is acknowledged by the receiving federation proxy. The federation gateway will continually attempt to establish a connection and reconnect after any connection failures (retrying with exponential backoff). When a connection is established, the federation gateway will send queued operations in order. The receiving federation proxy sends acknowledgements back to the sending federation gateway on a back channel and whenever an acknowledgement is received, the sender dequeues the acknowledged operations.



Example



Consider the case of a wavelet *W* with wavelet id (acmewave.com, conv+090528), where acmewave.com is a domain and “conv+090528” is an id string (whose structure does not concern us here). The wavelet id dictates that *W* is hosted by the AcmeWave wave provider. Suppose *W* has a participant feddy@federati.com from another domain federati.com.

All wavelet operations for *W*, submitted by local and remote participants alike, are transformed, applied to *W*, stored in the local wave store by the AcmeWave wave provider, and then the applied operations are passed to the federation gateway which pushes them to federati.com. The AcmeWave gateway does so by establishing a connection to the Federati federation proxy and sending the operations across the connection.

Sometimes the receiver needs to request past operations from the sender. The typical case is when it receives an operation for a wavelet where the receiver does not already possess all preceding operations for the wavelet. (This condition is easily verified because applied operations carry consecutive version numbers.) In this case the receiving federation proxy will connect to the domain that hosts the wavelet and request the past operations that it is missing. (One way that a wave server can develop such a gap in the operation history for a remote wavelet is when no participant from its domain participates in the wavelet, at time *t1*, and then later, at time *t2*, a participant from its domain is added to the wavelet. The host federation gateway responds by sending the new `AddParticipant` operation forwarding all ensuing new operations to the federation proxy, but the latter must itself turn around and request the prior operations.)

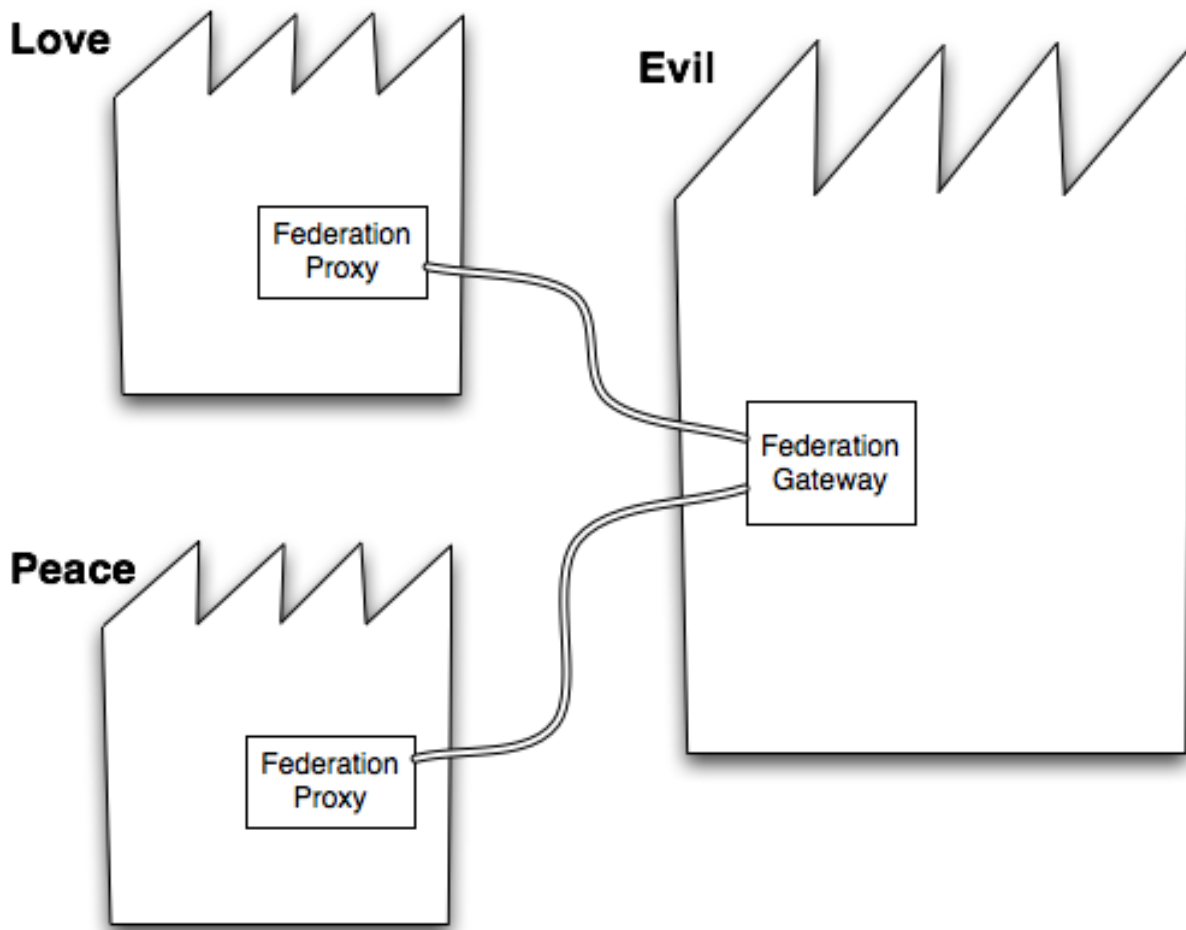
In the same way a user can submit operations to a remote wavelet, namely by letting the federation proxy connect to the remote federation proxy and submit the operation to its wave server.

Suppose there is another wavelet hosted by Federati, i.e., the wavelet id domain is federati.com, and this wavelet has a participant which is a user at acmeWave.com. Then the Federati gateway and AcmeWave gateway will also communicate with each other.

3.5 Protocol

The network protocol between federation gateways and proxies is called the Google Wave Federation Protocol. It is an open extension to the XMPP Internet Messaging protocol. Some of the key useful features of XMPP that the wave federation protocol uses are the discovery of IP addresses and ports, using SRV records, and TLS authentication and encryption of connections. See “Google Wave Federation Protocol”.

The XMPP transport encrypts operations at a transport level, so it only provides cryptographic security between servers that connect directly to each other. An additional layer of cryptography provides end-to-end authentication between wave providers using cryptographic signatures and certificates, allowing all wavelet providers to verify the properties of the operation. Specifically, a downstream wave provider can verify that the wave provider is not spoofing wavelet operations, namely, it cannot falsely claim (1) that a wavelet operation originated from a user on another wave provider or (2) that it was originated in a different context. This addresses the situation where two users from different, trustworthy wave providers, say love.com and peace.com, are participants of the a wavelet that is hosted on a malicious wave provider evil.com. The protocol requires love.com to sign its user’s operations with love.com’s certificate and peace.com to sign its user’s operations with peace.com’s certificate. These signatures travel with the operations and evil.com must host the signatures together with the operations. Furthermore, love.com and peace.com will verify the signatures of all the operations that evil.com forwards. This makes it impossible for evil.com to alter or spoof the content of the messages from the user of love.com which is shared with peace.com, and vice versa. All the signing and verification is done by the wave providers, not the client software of the end users.



The protocol specification requires that wave providers connecting using the federation protocol must authenticate using cryptographically secure TLS mechanisms. Moreover, it is recommended that they use TLS to encrypt the traffic between them. The client-server and federation protocols do not provide end-to-end authentication or encryption between end users. A wave provider should authenticate its end users and encryption of user connections is also recommended. In combination, secure connections between wave services and secure connections between users and their wave services offer a reasonable level of end-to-end security.

3.6 References

Jochen Bekmann, Michael Lancaster, Soren Lassen, David Wang: [Google Wave Data Model and Client-Server Protocol](#)

David Wang, Alex Mah: [Google Wave Operational Transformation](#)

Daniel Berlin: [Google Wave Federation Protocol](#)

Lea Kissner and Ben Laurie: [General Verifiable Federation](#)

GOOGLE WAVE CLIENT-SERVER PROTOCOL WHITEPAPER

Authors Joe Gregorio

Version 2.0 - May 2010

This whitepaper is part of a series. All of the whitepapers can be found on [Google Wave Federation Protocol site](http://google.com/wave-federation-protocol).

4.1 Editorial Notes

To provide feedback on this draft join the wave-protocol mailing list at <http://groups.google.com/group/wave-protocol>

This current draft only covers a small subset of the functionality that is required to build a full client. Future drafts will expand to cover more functionality.

4.2 Introduction

This document describes the protocol by which a wave client communicates with a wave server in order to create, read, and modify waves. The protocol is defined in terms of JSON messages exchanged over WebSockets.

4.3 Background

There is already a protocol being defined to handle the federation of Waves, however it was designed as a server-to-server protocol and is not well suited for clients. What is needed is a lighter weight protocol that only captures the needs of a client-server communication channel. The WebSockets protocol was chosen because it provides the two-way communication channel needed to efficiently handle wave messages, while being light weight and targeted to browsers, which are considered a primary platform for client developers.

4.4 Scope

This specification only covers the rudiments of the communication between a client and a server. There are many things that are not covered by this specification at this time, such as authentication, authorization, how a client determines which server to talk to, or which port to use. This protocol is a very simple client/server protocol implementation, and does not reflect the Google Wave web client protocol used in production today.

4.5 Data Model

It is important to understand the [Wave Federation Protocol](#) and [Conversation Model](#) as a prerequisite to this specification.

4.5.1 Terminology

The following terminology is used by this specification:

- wave - a collection of wavelets
- wavelet - a collection of named documents and participants, and the domain of operational transformation
- document - a structured wave document
- wave message - a single message sent either from the client to the server or from the server to the client.

Wave messages do not include the WebSocket opening handshake messages.

4.6 Operation

This section assumes an elementary understanding of the theory of [Operational Transforms](#).

4.6.1 Protocol Version

In the current implementation the version of the protocol is carried in each message and if the server does not understand the version sent it closes the connection. Future revisions may have the client and server negotiate for an agreed upon protocol version.

The version of the protocol used is 1.

4.6.2 Transport

The protocol begins when a Wave client connects with a Wave server. The connection is handled by the WebSockets protocol. After the connection is initiated Wave messages are sent between the client and server encapsulated in WebSocket frames. Each message occupies a single frame.

4.6.3 Transport Error Conditions

WebSocket Errors

TBD

Timeouts

TBD

Error recovery

TBD

4.6.4 Message Flow

There are two kinds of Wave requests, `ProtocolOpenRequest` and `ProtocolSubmitRequest`. Communication begins when a client sends a `ProtocolOpenRequest` to the server with the id of a Wave it wishes to monitor and/or mutate. After opening a wave the client may send `ProtocolSubmitRequests` to the server to manipulate the wave. The server will send `ProtocolWaveletUpdates` to the client as the server representation of the wave changes.

Any error messages related to the opening of a wave are sent back from the server in a `ProtocolWaveletUpdate`.

A client may send more than one `ProtocolOpenRequest`, one for each wave that the client is interested in.

The client **MUST** send a `ProtocolOpenRequest` for each wave that the client is interested in. A client **MUST NOT** send mutations for a wave id that it has not issued a `ProtocolOpenRequest` for. The client must wait for the server to acknowledge the `ProtocolOpenRequest` before sending `ProtocolSubmitRequests` for the given wave as it needs to include the document hash with each `ProtocolSubmitRequest`.

ProtocolOpenRequest

The `ProtocolOpenRequest` contains a wave id and a `wavelet_id_prefix`. Those two determine the set of wavelets that the client will be notified of changes to.

The `wavelet_id_prefix` may be shortened to match a larger subset of wavelets, with the empty string matching all wavelets in the given wave.

The client can indicate if it supports snapshots when it sends a `ProtocolOpenRequest`.

It also contains the protocol version number, which is defined as 1, per the previous section on Protocol Version.

ProtocolWaveletUpdate

In response to a `ProtocolOpenRequest` the server may send any number of `ProtocolWaveletUpdate` messages. The `ProtocolWaveletUpdate` may contain a snapshot of the current wave state or it will contain one or more `ProtocolWaveletDelta` messages that represent deltas to be applied to wavelets that the client is monitoring. The inclusion of the snapshot is determined by the server, it will only be sent on the first `ProtocolWaveletUpdate`, and will only be sent if the client has indicated in its `ProtocolOpenRequest` that it supports receiving snapshots.

`ProtocolWaveletUpdate` messages will only be sent for wavelets that the client is an explicit participant in.

ProtocolSubmitRequest

This message contains a `ProtocolWaveletDelta` which the client requests the server to apply to a wave. Only one submit per wavelet may be outstanding at any one time.

The client specifies which version to apply the delta at, and the client is expected to transform deltas pending for submission against deltas received in `ProtocolWaveletUpdates` from the server.

`ProtocolWaveletDelta`'s are applied atomically and either fully succeed, or the whole delta will fail.

ProtocolSubmitResponse

The `ProtocolSubmitResponse` acknowledges the `ProtocolSubmitRequest` and if the delta was successfully applied it also supplies the `ProtocolHashedVersion` of the wavelet after the delta, which the client will need to successfully submit future deltas to the wavelet.

Closing a wave

TBD

4.7 Specific Flows

4.7.1 Search

TBD

4.7.2 Creating a new wave

Creating a new wave is different from other flows since neither the client nor the server have the wave id. The client must generate a unique id for the wave and send a ProtocolOpenRequest for that wave id.

Entropy and Wave ID Length

TBD

4.8 Serializing Protocol Buffers as JSON

There is no standard serialization of Protocol Buffers into JSON. This section will define the serialization that is used to construct Wave Messages from the protocol buffers included in this specification.

Protocol buffer messages may be nested, so this serialization algorithm must be applied recursively.

The root level message is emitted as a JSON object. Each member of the message will be emitted as a key-value pair in the JSON object. Each member's key name in the JSON serialization is set to `normalize(key)`, where `normalize` is a function that takes in the protocol buffer member key name and returns a JSON utf-8 string.

4.8.1 `normalize()`

TBD

4.8.2 Member value serialization

The serialization of a value for the key is dependent on the type and modifiers of that member. If the member is flagged as 'repeated' then the serialized value will be a JSON array. The array will be filled with the serialized values of the repeated members.

4.8.3 Modifiers

The following modifiers can be applied to message values and they alter how the values are serialized.

repeated

For each repeated member value, serialize it as JSON according to the following rules and add the serialization to the JSON array.

required

Required parameters are always serialized into JSON.

optional

Optional parameters are only serialized if they appear in the protocol buffer.

4.8.4 string

A string member of a protocol buffer message is serialized as a JSON string.

4.8.5 int

An int32 or int64 member of a protocol buffer message is serialized as a JSON number.

4.8.6 bool

A bool value is serialized as a JSON number with a value of 1 for true and 0 for false.

4.8.7 enum

An enum value is serialized as a JSON string for the enumeration's value.

4.8.8 bytes

A bytes value is hex encoded and serialized as a JSON string.

4.8.9 message

A protocol buffer message is serialized by recursively applying the rules in this section.

4.9 Security

4.9.1 Securing the channel

TBD

4.9.2 Authenticating the client

TBD

4.9.3 Authorization

Authorization is covered in the [Access Control Whitepaper](#).

4.10 Client-Server Protocol Buffers

While the client server protocol is implemented as JSON over WebSockets, each Wave message is a JSON serialization of a protocol buffer. The protocol buffer definitions are defined as:

TBD

4.11 Example Client-Server Flow

TBD

4.12 Appendix A - Open Source Implementation Notes

The current open source implementation of the client-server protocol begins with the client opening the wave “indexwave!indexwave”. That is currently an implementation detail and is not documented.

ACCESS CONTROL IN GOOGLE WAVE

Authors Jon Tirsén

Version 1.0 - May 2009

Google Wave's primary means of access control is the list of addresses that participate on a wavelet and what access accounts has to these addresses. This white paper outlines how the wave platform stores, exchanges and enforces access control.

This whitepaper is part of a series. All of the whitepapers can be found on [Google Wave Federation Protocol site](#).

5.1 Executive summary

Wave access control is defined as:

- which individual or robot has access to a specific account,
- what access an account has to an address,
- and finally what access an address has to a wavelet (see [Google Wave Data Model and Client-Server Protocol](#) for more information on the wavelets).

Access from individuals to accounts and accounts to addresses is defined and enforced inside each wave provider and not specified in the standard. Address access is modeled as a graph where each edge in the graph grants access from one address to another address. These edges are stored in waves and are authorized by the wave provider that controls the domain of the addresses. The access edges can be exchanged between wave providers through the normal wave federation protocols.

Typically an account has access to a canonical address which is the entry point for an account into this graph, although this is wave provider specific. Operations are authorized at the source of each wave provider. If authorization spans multiple wave providers the operation needs to be sent and verified along the path of each of the involved wave providers. Different levels of access to a wavelet is still to be defined.

5.1.1 Accounts and addresses

Account An account belongs to an end user or a robot. Exactly how accounts work is wave provider specific. For example, Google uses Google Accounts to store and authenticate accounts, so a Google Wave account is shared with other Google properties.

Address Most of the system does not deal directly with accounts but rather with addresses. An address is a string formatted as an email address (RFC 2822). Addresses, rather than accounts, participate in wavelets.

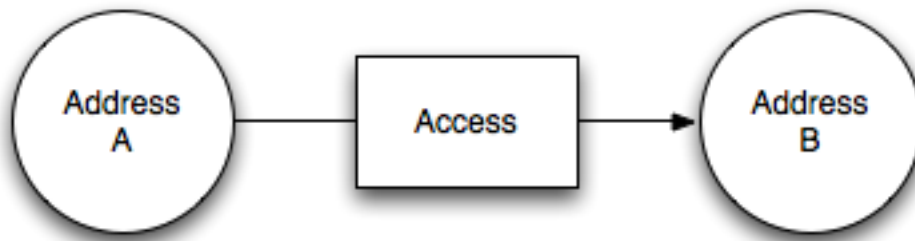
Canonical address Each account has a canonical address which is the address the user acts as normally. Most per-user metadata is stored with the canonical address as a key. The canonical address cannot generally be changed, but an account can participate on a single wavelet as multiple addresses.

5.1.2 Authentication

Each wave provider chooses how they authenticate their users. In Google Wave we use a simple username and password scheme for individuals. Robots are contacted by the Google Wave provider through a well-defined URL and are therefore authenticated that way.

5.1.3 Address access as a graph

Address access can be seen as a directed graph of address to address edges where each edge is restricted by access settings.



The entry point into the graph for a user or a robot is their canonical address.



There are multiple types of access which indicate what address A can do as address B.

- Indexed to (INDEX) - wavelets addressed to address B will be written into the index of the account associated with address A (transitively).
- Add (ADD) - address A can add address B to wavelets.
- Add myself as (ADD_ME) - address A can add address A to wavelets as address B.
- Read (READ) - address A can read wavelets addressed to address B.
- Write (WRITE) - address A can do anything as address B. This could also be called “act as”.
- Grant (GRANT) - address A can grant additional access edges to address B.

5.1.4 Storing and exchanging access edges

Wave representation of an access edge Access edges are stored in data documents in wavelets as follows:

```
<grant from="a@example.com" to="b@example.com" until="2009-06-14T13:31Z">
  <access>INDEX</access>
  <access>READ</access>
</grant>
```

Authorized access edge An authorized access edge is an access edge stored in a wave that can be attributed to an author that has a Grant access edge to the to address she is granting additional access too. This attribution should be enforced and verified by the wave provider. For example, the Google wave provider uses a namespace for all access edges. The namespace policy for that namespace will not allow edits that are not authorized.

Access wave Each account has an access wave identified by its canonical address. This contains the entire access sub-graph that is reachable from the canonical address. The wave provider of the account maintains this access wave by copying all the relevant and authorized access edges it encounters while indexing its own and its federated wave providers' waves. This means that access edges can be stored and distributed anywhere in the system as long as they are authorized as above. The storage and distribution mechanism of Google Wave itself is used to store and distribute this information.

Time to live When a wave provider issues an access edge to federated servers they are issued with a limited time period. They have to be refreshed within that time period or they are no longer valid. The time period should be chosen to minimize chattiness of the protocol and still allow for timely revocation. This is typically used for READ authorization when opening of a wavelet is not validated at the owning wave provider.

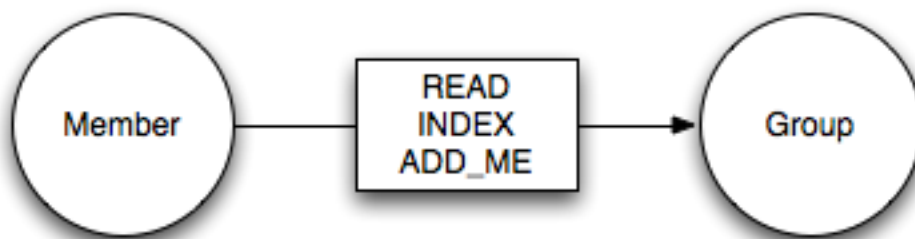
5.1.5 Authorizing an operation

An operation always contains the path of authorization from the canonical address to the address the account wants to perform an operation as excluding any initial WRITE edges. Using the information available in the access wave the client builds the path and inserts it into the operation that it sends to its wave provider. After it has optimistically applied the operation to the wave in the client it sends it to its wave provider who then signs and forwards the operation to the next wave provider in the path. Every wave provider on the path will validate and sign the operation before it is finally forwarded and applied at the wave provider that owns the wavelet. This final wave provider is responsible for verifying all the signatures.

If an authorization fails, the client has typically already optimistically applied the operation to the wave so will either need to reverse those operations or indicate an error to the user. In a well-behaved system this should only occur if an access edge has been removed or changed and this change has yet to be forwarded to the clients wave provider. In this case the client would access edges that are no longer valid.

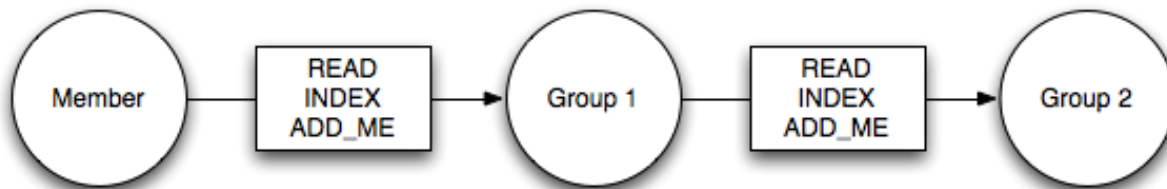
5.2 Groups

Groups are implemented on top of this generic access framework. Each group has an address and members. Group membership is expressed as the following edge for each member of the group:



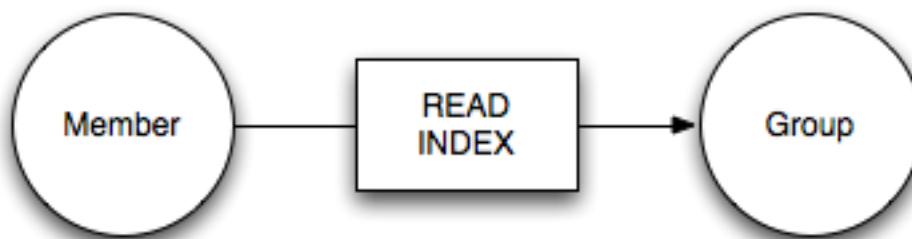
As you can see an important detail of groups in wave is that being a member of a group does not allow you to directly write into a wave which that group is addressed to. Instead it lets you add yourself as a direct participant to that wave.

A group can be a member of another group which looks like this:



This means that wavelets addressed to both Group 1 and Group 2 will be written into the member's index and the member can read and "write" (add self as a participant) to all these wavelets.

Read-only groups mean that the "add myself" access is lacking:



An address can be a read-write member of a nested group even though it's a read-only member of an outer group:



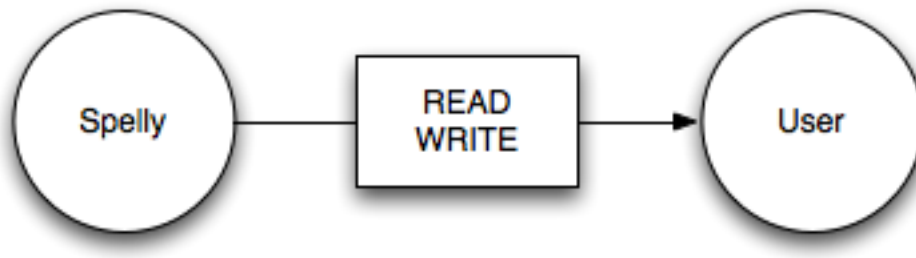
This means the member can become a participant of wavelets addressed to Group 1 but not to wavelets addressed to Group 2.

5.2.1 Delegation

Delegation allows an account to perform operations with another address as the author. Google Wave currently uses this for two cases:

- An account that is a write-member of a group can perform an AddParticipant operation to add an address belonging to that account to a wavelet.
- Google Wave's spelling ("Spelly"), linking ("Linky"), and other infrastructure services act on behalf of any address in a wavelet with those services enabled.

This last case is represented as the following edge:



This provides Google Wave infrastructure services full access to act as a user, while being authenticated as a service account.

5.2.2 Per-wavelet access control

Google Wave will eventually support some level of access control on a wavelets but requirements and implementation plans have yet to be determined. For example:

- A “commenter” role whereby a user can only create new blips and edit their own blips.
- A “confidential” mode (on the whole wavelet) or role (on a participant) where participants can’t add new participants.

GOOGLE WAVE ATTACHMENTS

Authors Michael Lancaster

Version 1.0 - May 2009

Wave messages may contain embedded binary attachments, such as images, PDF documents and ZIP archives. Because these binary files are qualitatively and quantitatively different to other wave content (rich text), they are handled as a somewhat special case within Google Wave. This document gives an overview on how attachments are represented within Google Wave, and how the servers interoperate to handle attachment uploading and serving.

This whitepaper is part of a series. All of the whitepapers can be found on [Google Wave Federation Protocol site](#).

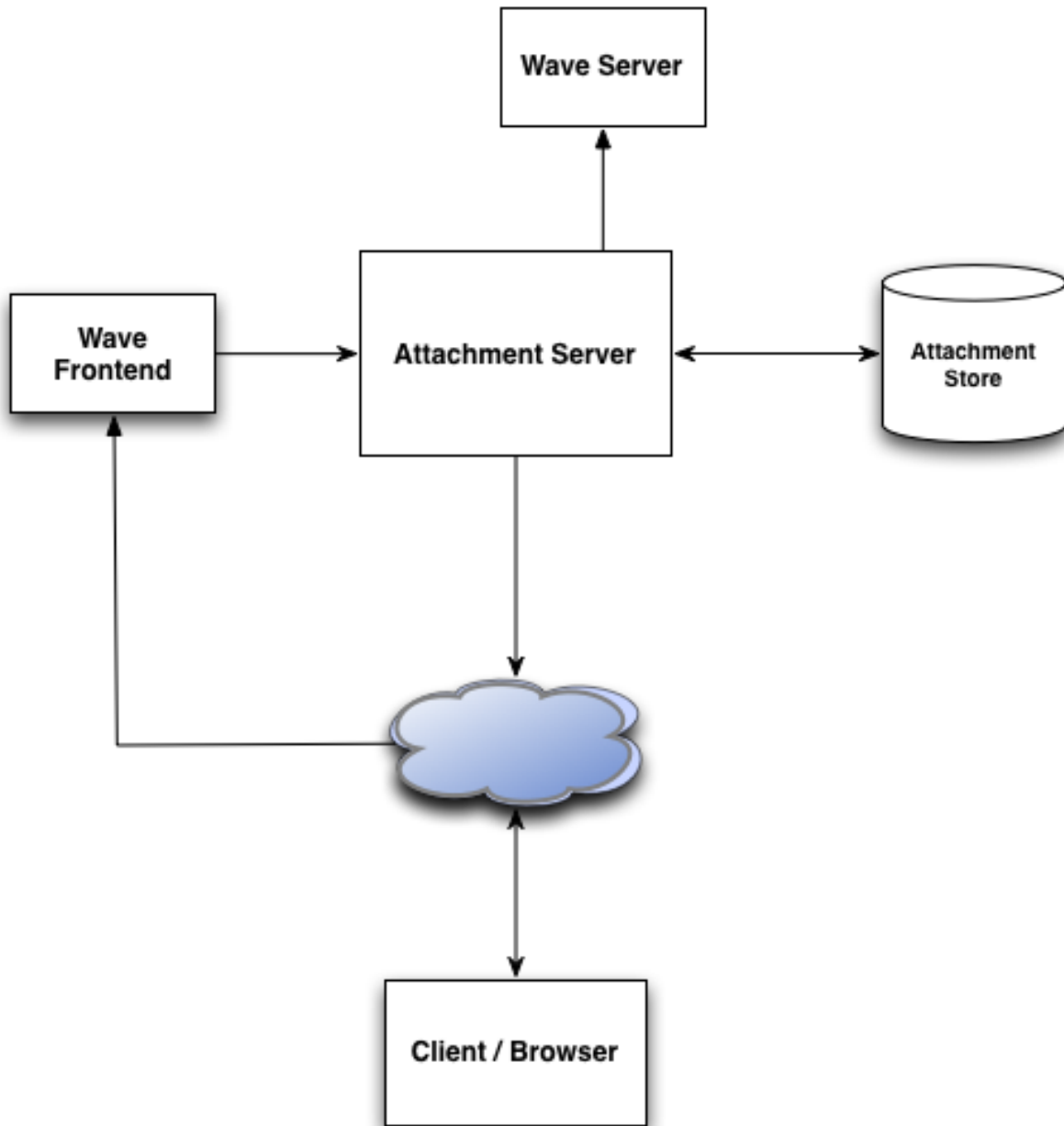
6.1 High level summary

Attachments are represented within a wave by an XML document, allowing changes (in upload progress, for instance) to be propagated to all users on the wave. Each attachment has a corresponding thumbnail image. For image attachments, this thumbnail is actually a small version of the image itself. In order to reduce latency for image attachments, HTML5 or Gears enabled clients may generate and upload a thumbnail before the image itself. For most other attachment types, the thumbnail is a generic representation of the attachment type (base on MIME type). Attachments are uploaded by the wave client using HTTP POST, and download of both attachments and their thumbnails is done using HTTP GET.

6.2 Architecture

Attachment management is handled by a dedicated attachment server. This server is responsible for handling create, upload and download requests, generating thumbnails, reprocessing images, malware scanning, as well as for communications with the attachment store.

The attachment server acts as an HTTP server (for handling attachment operations from the client), an RPC server (for handling attachment operations from internal agents, such as the mail gateway), and an RPC client for propagating attachment metadata to the wave server (see Google Wave Federation Architecture for details on the overall Google Wave architecture).



6.3 Schema

Each attachment has a globally unique ID string, composed of the wave service provider domain, and a string that is unique for that provider. An example attachment ID for the wave sandbox wave provider would be “wavesandbox.com/3eb1c8ba-172b-4b1a-ae5b-d3140ed85c42”. Each attachment is represented by a row in a replicated Bigtable (a Google proprietary scalable distributed database). The attachment metadata is represented by a protocol buffer stored in a column on that row. This protocol buffer contains such fields as the attachment size, upload progress, filename of the attachment, as well as a list of all wavelets that reference this attachment. The binary data for the thumbnail and attachment are each stored in separate columns on the same row.

Thus an attachment row in the Bigtable looks like:

AttachmentMetadata contains the metadata protocol buffer,

ThumbnailData used to store the thumbnail BLOB (binary large object),

AttachmentData used to store the attachment BLOB, for small attachments

Large attachments are stored in a separate Bigtable for better storage efficiency.

For performance, and simplicity of design, a subset of the attachment metadata is also copied to any wavelets which reference the attachment. Storing this metadata in the wavelet means that we don't have to do anything special at wave load time to ensure that the client has a copy of the attachment metadata. Whenever the attachment server makes a modification to the attachment metadata, it pushes out the change to all relevant wavelets (via RPC to the wave server(s)).

This copy of the metadata is represented by an XML sub-document on a data document within the wavelet. The ID of the Data Document is based on the attachment ID such that there is exactly one attachment data document for each attachment on a given wavelet.

The attachment metadata XML sub-document is defined by the following RNC (Relax NG) schema:

```

element attachment {
  attribute attachmentId { text },
  attribute uploadProgress { xsd:integer },
  attribute attachmentSize { xsd:integer },
  attribute malware{ 0, 1 },
  attribute stalled { 0, 1 }? // default = 0
  attribute filename { text },
  attribute mimeType { text },
  attribute downloadToken { text },
  element thumbnail {
    attribute width { xsd:integer },
    attribute height { xsd:integer },
    attribute clientGenerated { 0, 1 }? // default=0
  }?
  element image {
    attribute width { xsd:integer },
    attribute height { xsd:integer }
  }?
}

```

Changes to the attachment record are replicated to all waves which refer to that attachment.

The blip in which the attachment was inserted also contains an XML node which references the attachment, located at the insertion point of the attachment. This XML element (known as the embed) is a placeholder for the thumbnail to be rendered and takes the form:

```
<w:image attachment="attachment id"><w:caption>the thumbnail caption</w:caption></w:image>
```

6.4 Attachment Creation

Attachments may be “created” in several different ways:

- Uploading a thumbnail for the attachment
- Uploading the attachment blob itself (or the first N bytes)
- Linking an existing attachment to a new wave

Each of these actions is represented by an attachment creation request. Attachment creation requests are sent as an HTTP POST, and may be either sent as an HTTP multipart request (enctype=multipart/form-data), or as a plain POST

(`enctype=application/x-www-form-urlencoded`). The multipart POST is accepted to allow file uploads from non-Gears / HTML5 enabled browsers.

In either case, the following fields may be sent either as HTTP POST parameters, or in the HTTP header:

```
required string attachmentId;
required string waveletName;
required int uploadType; // 0 for attachment, 1 for thumbnail
optional bool complete; // true if data field represents the entire attachment
optional int thumbnail_width;
optional int thumbnail_height;
```

For the non-multipart case, the filename is also optionally provided in the parameters / header.:

```
optional string fileName;
```

and the bytes of the attachment / thumbnail are sent as the body of the POST.

In the multipart case, only the part with name set to “uploadAttachment” is read, any other uploaded files are ignored. The filename is read from the filename field in the content-disposition for the file.

Create requests are idempotent, so for instance it’s okay to send one creation request with a thumbnail, and another with the first chunk of the attachment data. If the attachment record already exists, but the waveletName field does not correspond to any of the wavelets currently linked to the attachment, the existing attachment will be linked to the provided wavelet. Other fields which are already present in the existing attachment will be ignored.

Example creation flow:

1. User initiates attachment creation by dragging an image into the browser (using Gears)
2. Client generates a globally unique ID for the attachment
3. Client thumbnails the image (using Gears) and displays it locally by adding an `<image>` tag to the blip (other clients seeing the `<image>` tag will display an empty thumbnail frame). The client then sends an HTTP POST containing a create request, and the thumbnail data, to the Attachment server (via WFE)
4. Attachment server creates a record in permanent storage for the attachment and stores the (re-encoded for security) user-provided thumbnail
5. Attachment server returns success to the client
6. Attachment server creates a data document on the wavelet and adds a copy of the attachment metadata.
7. Thumbnail is now ready to download
8. Client sends an HTTP POST containing the attachment
9. Attachment server updates the attachment record in permanent storage
10. Attachment server returns success to the client
11. Attachment server generates a thumbnail for the attachment
12. Image attachments are reprocessed to prevent XSS attacks, and attachments are scanned for malware
13. Attachment server updates the attachment data document on the wavelet
14. Attachment is now ready to download

Steps 8-14 may happen in parallel with 3-7.

Below is an example of a multipart (non-Gears) creation request:

```
POST /wfe/upload/result HTTP/1.1
Host: wave.google.com
Content-Type: multipart/form-data; boundary=-----10102754414578508781458777923
```

```
Content-Length: 195197
-----10102754414578508781458777923
Content-Disposition: form-data; name="uploadAttachment"; filename="Downtown.pdf"
Content-Type: application/pdf

<encoded attachment binary data here>

-----10102754414578508781458777923
Content-Disposition: form-data; name="waveletName"

wavesandbox.com/w+6bf32acc-bd29-45c2-a252-699af690f5a6/conv+root
-----10102754414578508781458777923
Content-Disposition: form-data; name="attachmentId"

wavesandbox.com/3eb1c8ba-172b-4b1a-ae5b-d3140ed85c42
-----10102754414578508781458777923
Content-Disposition: form-data; name="uploadType"

0
-----10102754414578508781458777923--
```

6.5 Uploading

Clients may upload large attachments in multiple chunks using an upload request:

```
required string attachmentId;
required int offset;
optional int fullSize;
```

The binary data is sent as per the creation request. Either multipart or form POSTs are accepted.

An upload request may not be sent until the upload request (or create request) for the previous chunk has been acknowledged. That is, we don't currently support pipelining. Chunks must not overlap. Behaviour is not specified if chunk boundaries overlap.

The response to HTTP upload / create requests is a string containing a single JSON object of the form:

```
{
  responseCode: <response>,
  errorMessage: "<error message> "
}
```

Possible values for the responseCode field are:

```
0 (OK)
1 (INVALID_TOKEN)
2 (INVALID_REQUEST)
1000 (INTERNAL_SERVER_ERROR)
```

The errorMessage field will not be provided for the non-error case (OK). Otherwise, it will contain a human-readable (although not necessarily end-user friendly) error message.

In conjunction with these custom error codes, HTTP response codes should also be respected, however, due to limitations with cross-domain POSTs, the JSON response codes are used in preference.

6.6 Attachment / Thumbnail download

A download request takes the following form:

```
required string attachmentId;  
required string downloadToken;
```

Requests for thumbnails / attachments are sent on different URLs, but otherwise look identical.

The response to these requests is an HTTP response containing the bytes of the attachment / thumbnail, with the HTTP Content-Disposition header set to “attachment”. The mime type of the response is set to the mime type of the attachment or thumbnail.

6.7 Authentication / Authorization

Google web-apps use a centralized cookie-based authentication system. Authentication for upload and creation requests uses this system. In order to write the corresponding attachment data document into an associated wavelet, the user must be a participant on that wavelet.

Downloads are authenticated using a download token which is stored in the attachment data document on the wavelet. Thus to download an attachment or a thumbnail, the user must at some point in time have had access to both the attachment id and the download token.

6.8 Duplicate elimination

Because we expect a large percentage of attachments to be duplicates, we have an offline de-duping procedure. We store a weak hash with each attachment, and an offline process indexes attachments by hash, detects collisions, and then does a byte-by-byte comparison to eliminate duplicates. This is only done on attachments that are completely uploaded, and effectively immutable, and only on ‘large’ blobs, which are stored in a separate store. We maintain a level of indirection for these large blobs, so that we don’t have to update the pointers upon duplicate detection and to prevent the leakage of information about the existence of previously uploaded attachments.

6.9 References

- 5. Nebel and L. Masinter, [Form-based File Upload in HTML](#), IETF RFC 1867, November 1995
- 6. Chang et al., [Google Research Publication: Bigtable](#), OSDI’06: Seventh Symposium on Operating System Design and Implementation, November 2006.
- 19. Lassen and S. Thorogood, [Google Wave Federation Architecture](#), June 2009